

Lecture 5: Advanced Relational Algebra

Amit Kumar Dhar

August 7, 2025

Introduction

Our Running Example: Student Database

Students(sid,sname,discip)

sid	sname	discip
101	Isha	CSE
102	Parth	DSAI
103	Juhi	CSE
104	Suresh	MA

Enrolled(sid,cid,grade)

sid	cid	grade
101	CSL303	A
102	DSL201	B
103	CSL303	B
101	CSL211	A

Other Operations

Renaming (ρ)

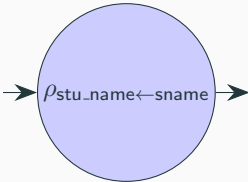
Concept: Renames attributes or the entire relation.

Example: Rename the 'sname' attribute to 'stu_name'.

Formula:

$$\rho_{\text{stu_name} \leftarrow \text{sname}}(\text{Students})$$

sid	sname	discip
101	Isha	CSE
102	Parth	DSAI



sid	stu_name	discip
101	Isha	CSE
102	Parth	DSAI

Duplicate Elimination (δ)

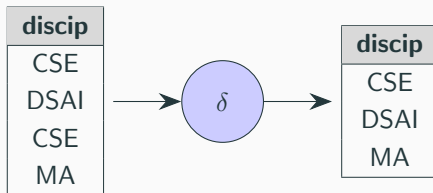
Concept: Explicitly removes duplicate rows from a relation. While standard projection (π) implicitly does this, δ can be used on any relation.

Example: Get the list of all discs, ensuring each appears only once.

Formula:

$$\delta(\pi_{\text{discip}}(\text{Students}))$$

$$\pi_{\text{discip}}(\text{Students})$$



Grouping & Aggregation (γ)

Concept: Groups rows based on some attributes and then computes an aggregate function on each group.

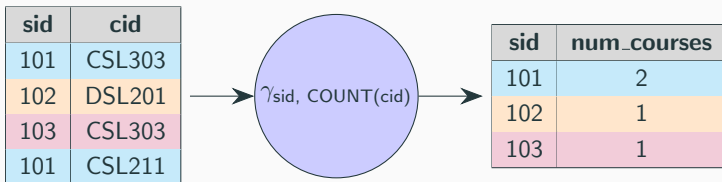
Formula Syntax:

$$\gamma_{\text{grouping_attributes}, \text{AGG}(\text{attribute})}(\text{Relation})$$

Common Aggregates: COUNT, SUM, AVG, MIN, MAX.

Example 1: Find the number of courses each student is enrolled in.

$$\gamma_{\text{sid}, \text{COUNT}(\text{cid}) \text{ AS num_courses}}(\text{Enrolled})$$



Grouping & Aggregation (γ)

Concept: Groups rows based on some attributes and then computes an aggregate function on each group.

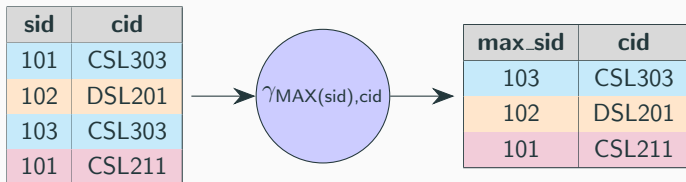
Formula Syntax:

$$\gamma_{\text{grouping_attributes}, \text{AGG}(\text{attribute})}(\text{Relation})$$

Common Aggregates: COUNT, SUM, AVG, MIN, MAX.

Example 2: Find the maximum among the sid registered for each course.

$$\gamma_{\text{MAX}(\text{sid}) \text{ AS } \text{max_sid}, \text{cid}}(\text{Enrolled})$$



Sorting (τ)

Concept: Sorts the tuples of a relation based on one or more attributes.

Formula Syntax:

$$\tau_{\text{attribute_list}}(\text{Relation})$$

Example: List students sorted by discip, then by name.

$$\tau_{\text{discip, sname}}(\text{Students})$$

Result of $\tau_{\text{discip, sname}}(\text{Students})$

sid	sname	discip
101	Isha	CSE
103	Juhi	CSE
102	Parth	DSAI
104	Suresh	MA

Query Representation

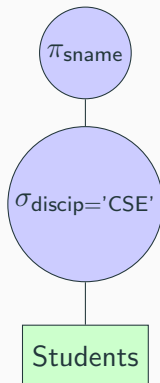
Relational Algebra to Query Trees

Every relational algebra expression can be represented as a tree:

- Leaves are relations (tables).
- Internal nodes are operators.
- The tree is evaluated from the leaves up.

Expression:

$\pi_{\text{name}}(\sigma_{\text{discip}='CSE'}(\text{Students}))$



Query Plan Transformation

Different trees can represent the same query. The query optimizer's job is to find the most efficient tree (the best "query plan").

A common heuristic: **Push selections down the tree.**

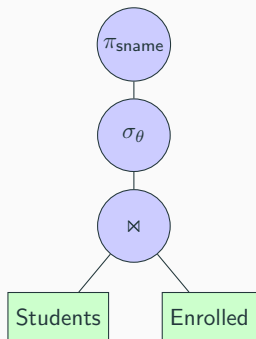
Query: Find names of CSE students enrolled in CSL303.

Query Plan Transformation

Query: Find names of CSE students enrolled in CSL303.

Inefficient Plan

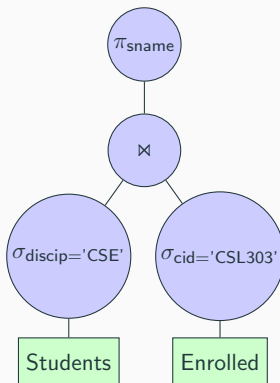
$\pi_{\text{name}}(\sigma_{\text{discip}='CSE' \wedge \text{cid}='CSL303'}(\text{Students} \bowtie \text{Enrolled}))$



(Join first, then select)

Efficient Plan

$\pi_{\text{name}}(\sigma_{\text{discip}='CSE'}(\text{Students}) \bowtie \sigma_{\text{cid}='CSL303'}(\text{Enrolled}))$



Complex Queries

Complex Query 1: Average Grade

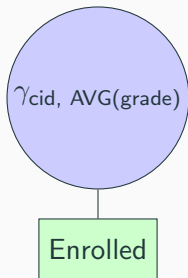
Query: Find the average grade for each course that has at least one student.

Complex Query 1: Average Grade

Query: Find the average grade for each course that has at least one student. **Expression:**

$\gamma_{cid, AVG(grade)} AS avg_grade(Enrolled)$

Query Tree:



Complex Query 2: CSE/DSAI Students with 'A's

Query: Find the names of students in 'CSE' or 'DSAI' who got 'A'.

Expression:

$\pi_{\text{name}}((\sigma_{\text{discip}=\text{'CSE'} \vee \text{discip}=\text{'DSAI'}}(\text{Students})) \bowtie (\sigma_{\text{grade}=\text{'A'}}(\text{Enrolled})))$

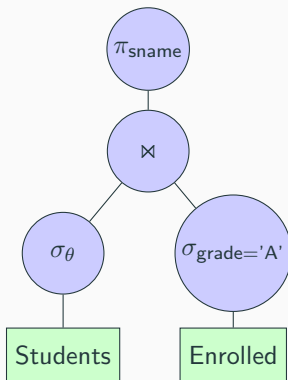
Complex Query 2: CSE/DSAI Students with 'A's

Query: Find the names of students in 'CSE' or 'DSAI' who got 'A'.

Expression:

$\pi_{\text{name}}((\sigma_{\text{discip}=\text{'CSE'} \vee \text{discip}=\text{'DSAI'}}(\text{Students})) \bowtie (\sigma_{\text{grade}=\text{'A'}}(\text{Enrolled})))$

Query Tree:



Complex Query 3: Popular Courses

Query: Find the 'cid' of courses with more than 1 student.

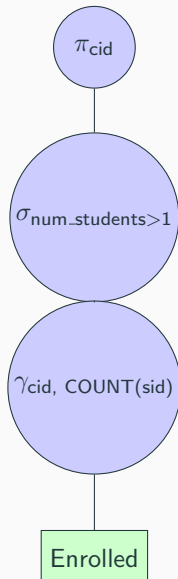
Complex Query 3: Popular Courses

Query: Find the 'cid' of courses with more than 1 student.

Expression:

```
 $\pi_{cid}(\sigma_{num\_students > 1}(\gamma_{cid, COUNT(sid) AS num\_students}(Enrolled)))$ 
```

Query Tree:



Complex Query 4: All-Seeing Eye

Query: Find the names of students who are enrolled in *all* courses that 'Isha' is enrolled in. (Relational Division)

Complex Query 4: All-Seeing Eye

Query: Find the names of students who are enrolled in *all* courses that 'Isha' is enrolled in. (Relational Division)

Expression:

1. Courses Isha is in:

$$C_A \leftarrow \pi_{\text{cid}}(\sigma_{\text{sname}='Isha'}(\text{Students} \bowtie \text{Enrolled}))$$

2. All (student, course) pairs: $SC \leftarrow \pi_{\text{sid}, \text{cid}}(\text{Enrolled})$

3. Students who are missing a course from C_A :

$$T_1 \leftarrow \pi_{\text{sid}}((\pi_{\text{sid}}(SC) \times C_A) - SC)$$

4. Students who are NOT missing any course:

$$T_2 \leftarrow \pi_{\text{sid}}(\text{Students}) - T_1$$

5. Final names: $\pi_{\text{sname}}(T_2 \bowtie \text{Students})$

Note: This is complex! Division is often expressed this way with fundamental operators.

Thank You!
Questions?